

Optimism Governor Audit



August 4, 2025

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
Security Model and Trust Assumptions	5
Privileged Roles	5
Low Severity	7
L-01 Deployment Script Incorrectly Calls updateTimelock Function	7
L-02 Incorrect Documentation	7
L-03 Incomplete Documentation	7
L-04 Possible Duplicate Event Emissions	8
L-05 Timelock Proxy Deployment and Initialization Split into Separate Transactions	8
L-06 OptimismGovernor Version Not Increased	8
L-07 updateTimelock Lacks Zero-Address Validation	9
L-08 proposeWithModule Requires Timelock to Have Proposal Threshold	9
Notes & Additional Information	10
N-01 Discrepancies between Documentation and Implementation	10
N-02 Repeated Calls to Grant Role	10
N-03 Commented-Out Code	11
N-04 Inconsistency in Indexing Event Parameters	11
Conclusion	12

Summary

Type	Governance	Total Issues	12 (12 resolved)
Timeline	From 2025-07-16 To 2025-07-18	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	8 (8 resolved)
		Notes & Additional Information	4 (4 resolved)

Scope

OpenZeppelin audited the following components from the [voteagora/optimism-governor](#) repository at commit [d585692](#):

- The changes made to `src/OptimismGovernor.sol` in [pull request #44](#)
- The entire `script/RedeployTimelock.s.sol` script
- The provided calldata for the proposal on the Sepolia network

System Overview

The `OptimismGovernor` contract introduces new logic for an authorized proposer, who is permitted to create proposals using the `propose` and `proposeWithModule` functions. In addition, a dedicated `setAuthorizedProposer` function has been added that can only be called by the manager or the timelock contract to set the authorized proposer's address.

The timelock redeployment script, `RedeployTimelock.s.sol`, implements the logic for the deployment of a new timelock contract using the transparent proxy pattern, initializing its state, and setting up roles and privileged accounts. The final part of the scope covers the calldata prepared for a proposal to invoke the governor's `updateTimelock` function with the address of the new timelock contract.

Security Model and Trust Assumptions

During the audit, the following observations and trust assumptions were made:

- The existing logic of the `OptimismGovernor` contract works as expected and does not contain issues, as only the introduced changes were reviewed in this audit.
- Using the `RedeployTimelock.s.sol` script, the deployer correctly sets the addresses for `EXISTING_GOVERNOR`, `MANAGER_ADDRESS`, `PROXY_ADMIN`, `L2_SAFE_1`, `L2_SAFE_2`, and `L2_SAFE_3`.

Privileged Roles

The changes made to the `OptimismGovernor` contract introduce a new privileged role, `authorizedProposer`, that is able to propose new proposals and cancel its own proposals. In addition, the timelock redeployment script (`RedeployTimelock.s.sol`) introduces `L2_SAFE_1`, `L2_SAFE_2`, `L2_SAFE_3` addresses that can cancel any proposal.

Update: During the fix review process, the `timelock` contract was removed from the authorized proposer set in the `OptimismGovernor` contract. Additionally, in the `RedeployTimelock.s.sol` script, `MANAGER_ADDRESS` was removed from the list of `CANCELLER_ROLE` addresses because it is anticipated that one of the SAFE addresses will be assigned to the `manager` address.

Low Severity

L-01 Deployment Script Incorrectly Calls `updateTimelock` Function

The `RedeployTimelock.s.sol` script contains a `call` to `governor.updateTimelock(newTimelock)` that will fail during execution due to access-control restrictions. The `updateTimelock` function of the `OptimismGovernor` contract has the `onlyGovernance` modifier, which requires `_msgSender()` to be the governor contract itself. However, when the script runs, the `_msgSender()` is the deployer's address, not the governor contract, causing the check to fail.

Consider removing the `updateTimelock` call from the deployment script.

Update: Resolved in [pull request #47](#) at [commit ce323b5](#).

L-02 Incorrect Documentation

The docstrings above the `propose` and `proposeWithModule` functions incorrectly state that an address with votes above the proposal threshold can propose. They also fail to mention the timelock contract as one of the proposers. Given that these functions are decorated with the `onlyValidProposer` modifier, only the manager, authorized proposer, or the timelock contract can call these functions.

Consider correcting the aforementioned instances of incorrect documentation.

Update: Resolved in [pull request #48](#) at [commit ef27aaf](#).

L-03 Incomplete Documentation

[This comment](#) in the `propose` function mentions that only the manager or timelock can propose. However, it does not mention the `authorizedProposer` address that can also propose.

Consider updating the comment to accurately reflect the complete set of valid proposers.

Update: Resolved in [pull request #48](#) at [commit ef27aaf](#).

L-04 Possible Duplicate Event Emissions

When a setter function does not check if the value being set is different from the existing one, it becomes possible to set the same value repeatedly, creating a possibility for event spamming. Repeated emission of identical events can also confuse off-chain clients.

The `setAuthorizedProposer` function sets the `authorizedProposer` variable and emits an event without checking if the value has changed.

Consider adding a check that reverts the transaction if the value being set is the same as the existing one.

Update: Resolved in [pull request #53](#) at [commit 68d60ba](#).

L-05 Timelock Proxy Deployment and Initialization Split into Separate Transactions

The deployment logic for the `Timelock` contract uses the transparent proxy pattern via `TransparentUpgradeableProxy`, followed by a separate call to the `initialize` function. This unnecessarily splits the deployment and initialization of the proxy into two distinct transactions, potentially exposing the contract to front-running attacks.

Consider encoding the initialization call in the `_data` parameter of the `TransparentUpgradeableProxy` constructor. This approach ensures that both the proxy and its implementation are initialized atomically within a single transaction.

Update: Resolved in [pull request #54](#) at [commit 600c183](#).

L-06 OptimismGovernor Version Not Increased

The `OptimismGovernor` contract was modified to include new logic for the authorized proposer. However, the `VERSION` was not incremented. As a result, the `reinitialize` function is not callable and will revert due to the `reinitializer` modifier. While it is possible to set the `authorizedProposer` using the `setAuthorizedProposer` function, this approach bypasses the standard practice of handling state changes during upgrades.

Consider increasing the `VERSION` of the contract and setting the parameters through the `reinitialize` logic.

Update: Resolved in [pull request #49](#) at [commit 01d6170](#). During the upgrade process, it is expected that the governor contract is upgraded and reinitialized atomically to prevent any front-running.

L-07 `updateTimelock` Lacks Zero-Address Validation

The `updateTimelock` function of the `OptimismGovernor` contract lacks zero-address validation for the `newTimelock` input. If `address(0)` is passed as `newTimelock`, the `OptimismGovernor` functions will become unusable. This is because the `onlyGovernance` modifier will block further updates, effectively bricking the contract's governance functionality.

Consider adding a zero-address check before assigning the `_timelock` state variable.

Update: Resolved in [pull request #56](#) at [commit 7d23fbc](#).

L-08 `proposeWithModule` Requires Timelock to Have Proposal Threshold

The `proposeWithModule` function is marked with the `onlyValidProposer` modifier, which allows the function to be executed by one of the following addresses: `manager`, `authorizedProposer`, or `timelock`. However, within the function, there is a check that requires the following: if the function is called by an address that is neither `manager` nor `authorizedProposer`, a [proposal votes threshold needs to be met](#).

Given that the only other address allowed to call the `proposeWithModule` function is `timelock`, the aforementioned check implies that `timelock` itself must meet a vote threshold. This design is inconsistent with the `propose` function where the check for privileged proposer addresses to meet the proposal threshold is [intentionally removed](#). In addition, this discrepancy raises a further discussion on whether the timelock contract should be allowed to be a proposer on the governor. Timelocks are typically meant to delay, queue, and execute proposals. Allowing the timelock contract to propose can create circular governance paths, which can prove to be error-prone.

Consider unifying the behavior for the proposal-threshold check across the `propose` and `proposeWithModule` functions. Moreover, reconsider the design decision of allowing the timelock contract to be a proposer.

Update: Resolved in [pull request #50](#) at [commit c40620f](#). The `timelock` contract is no longer a valid proposer.

Notes & Additional Information

N-01 Discrepancies between Documentation and Implementation

The `RedeployTimelock` deployment script sets up the `timelock` controller with parameters that are meant to reflect those described in the [scoping document](#). However, some discrepancies exist between the script and the documentation:

- The document mentions that "Cancellation rights over scheduled timelock calls are held exclusively by 4 addresses: 3 Optimism Foundation addresses provided in script and governor". However, in the script, there is an additional address, `MANAGER_ADDRESS`, that is [granted CANCELLER_ROLE](#).
- As per the document, the timelock has a delay of 7 `days`. However, `MIN_DELAY` in the script is `6 days`.

Consider updating the documentation to reflect the current implementation decisions.

Update: Resolved in [pull request #55](#) at [commit 20e632d](#). The Agora team stated:

This has been updated to reflect the document. One thing to note is that one of the Safe addresses will be the manager. Therefore, we do not need to redundantly grant the role.

N-02 Repeated Calls to Grant Role

In the `RedeployTimelock` script, the same roles are being granted to the `timelock` and `EXISTING_GOVERNOR` addresses multiple times: first during the [timelock initialization](#) and then in the [_setupRoles function](#).

During the timelock initialization, the `__setupRole` function of the `TimelockControllerUpgradeable` contract is called inside the `__TimelockController_init` call chain, and the `TIMELOCK_ADMIN_ROLE` role is granted to the `timelock` and `deployer` addresses. In addition, the `PROPOSER_ROLE` and `CANCELLER_ROLE` roles are granted to the `EXISTING_GOVERNOR` address. After the deployment of the timelock contract, the script calls an internal `__setupRoles` function that again grants the `TIMELOCK_ADMIN_ROLE` role to the `timelock` address, and the `PROPOSER_ROLE` and `CANCELLER_ROLE` roles to `EXISTING_GOVERNOR` address.

Consider removing the aforementioned duplication of roles in order to improve code clarity and maintainability.

Update: Resolved in [pull request #52](#) at [commit e35f3c7](#).

N-03 Commented-Out Code

The `propose` function contains [commented-out code](#) which negatively affects the readability of the codebase.

Consider removing the commented-out code and leaving a code comment explaining the decision of removing the threshold check.

Update: Resolved in [pull request #50](#) at [commit c40620f](#).

N-04 Inconsistency in Indexing Event Parameters

The `OptimismGovernor` contract defines several events that are emitted when privileged addresses are updated, such as `ManagerSet`, `AuthorizedProposerSet`, and `TimelockChange`. Among these events, only the first two use indexed parameters, while the `TimelockChange` event does not.

Consider evaluating whether the parameters in these events are intended to be indexed and making the use of indexed parameters consistent.

Update: Resolved in [pull request #51](#) at [commit 072970a](#).

Conclusion

The review covered updates to the `OptimismGovernor` contract, the timelock redeployment script, and the calldata for updating the governor's timelock on the Sepolia network. Key changes included introducing an authorized proposer mechanism, restricting proposer permissions, securely redeploying the timelock with proper initialization, and preparing the calldata to update the timelock reference in governance.

During the audit, several minor issues were identified and recommendations were made to improve code consistency and readability. The Agora team was highly cooperative and provided clear explanations throughout the audit process.